

OST
Ostschweizer
Fachhochschule

Mühlespiel Software

Team 2

Silvio Bellini, Christian Sutter, Ryan Mikus

Departement / Abteilung

WO WISSEN WIRKT.

Inhalt

- Architektur
- KI-Gegner
- Benutzeroberfläche
- Fazit
- Demonstration



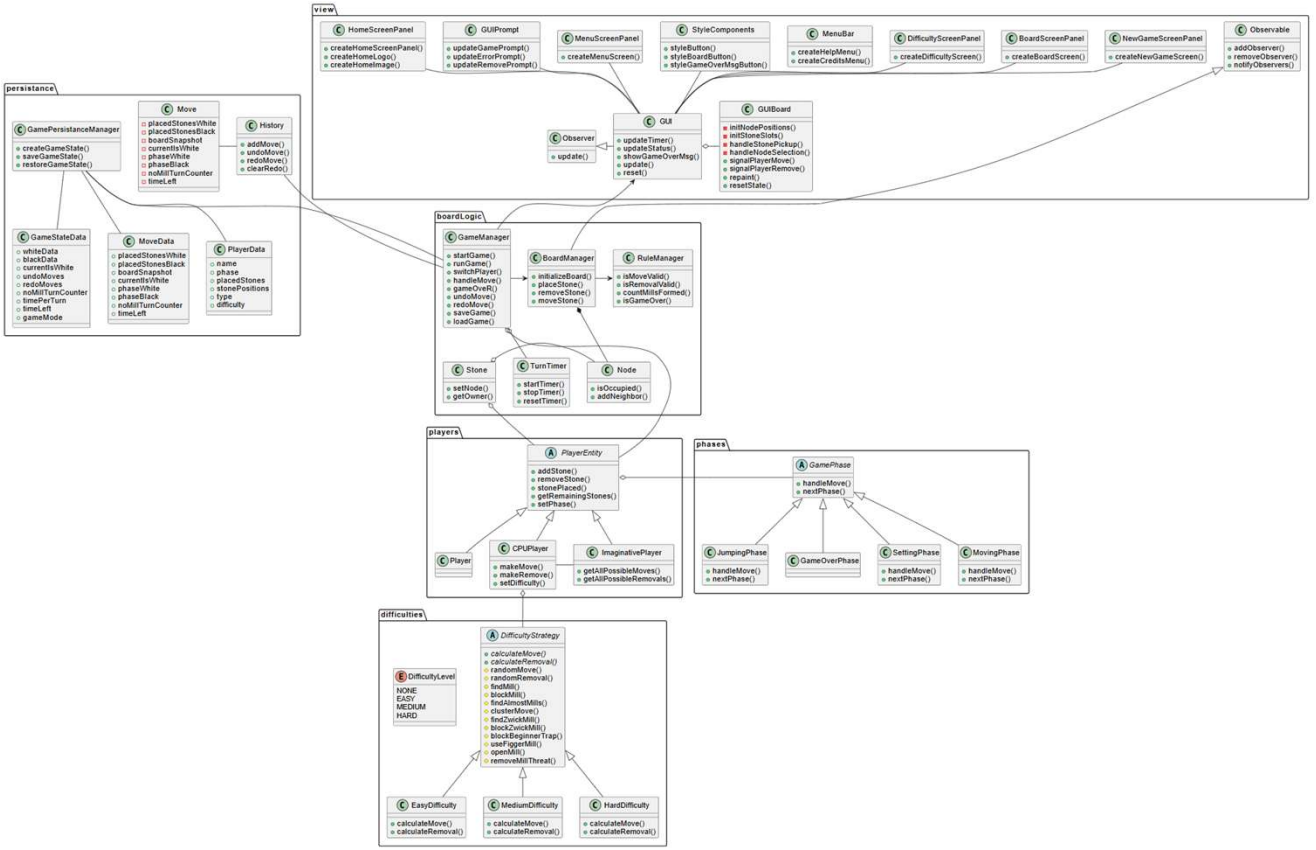
OST
Ostschweizer
Fachhochschule

Architektur

WO WISSEN WIRKT.

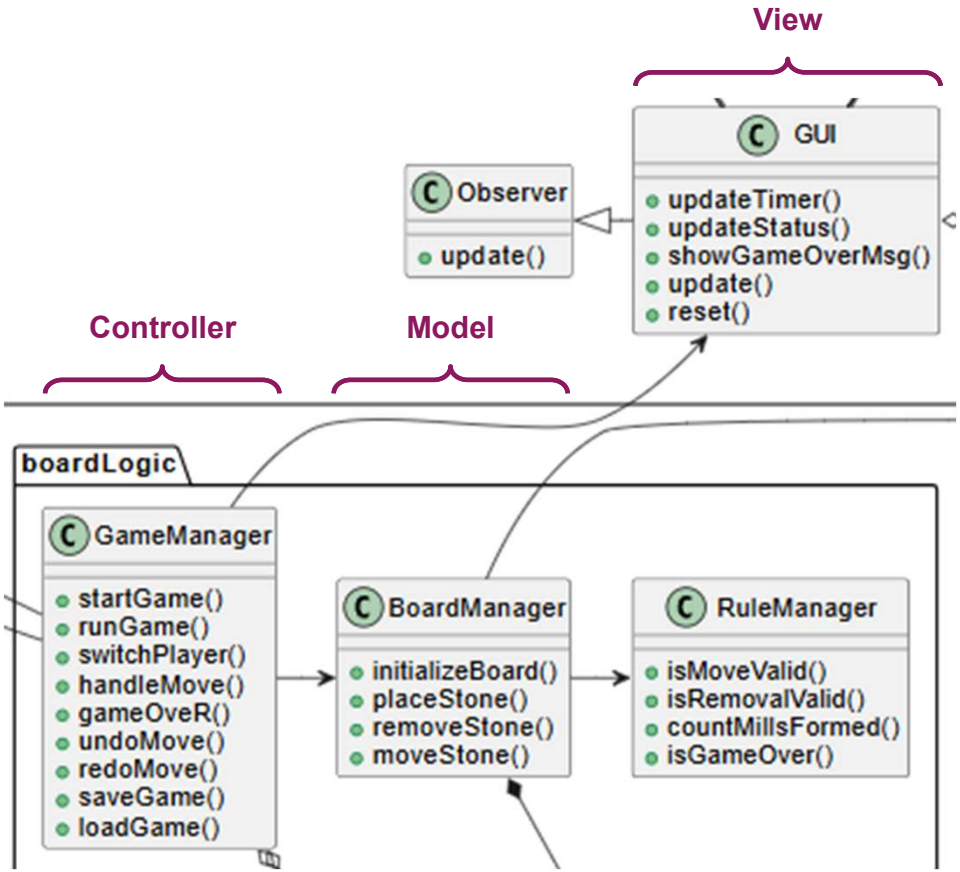
Architektur

Übersicht (Vereinfacht)



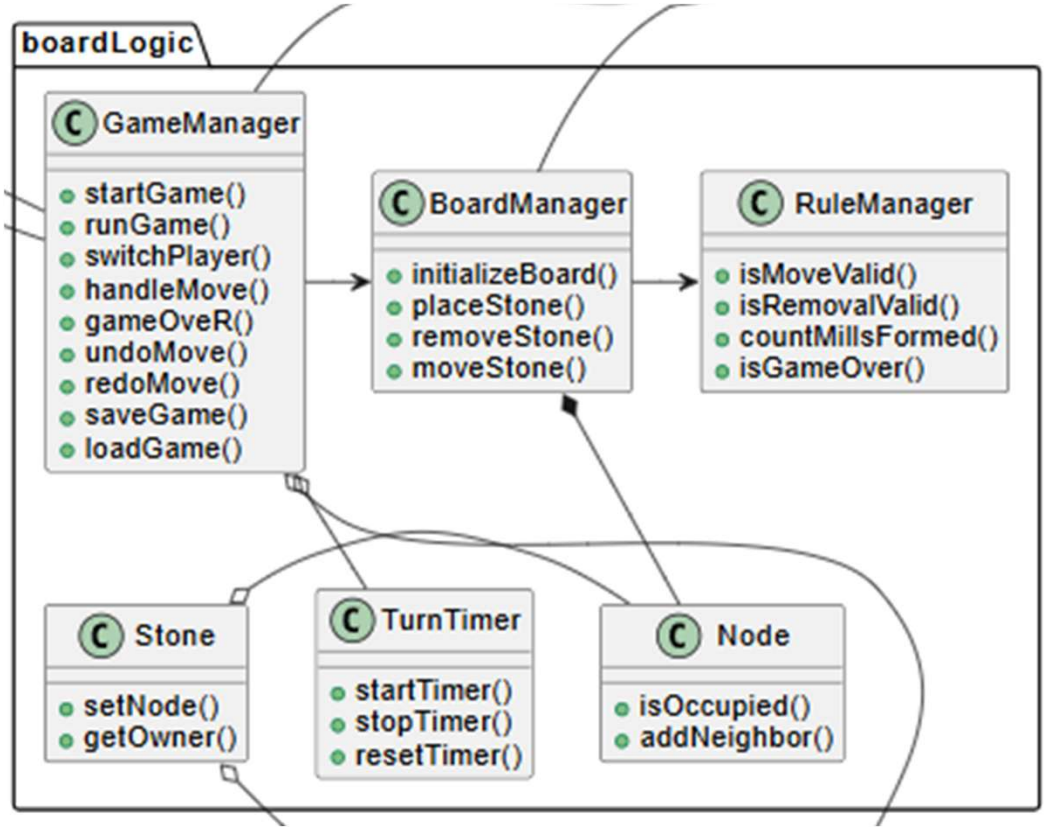
Architektur

MVC



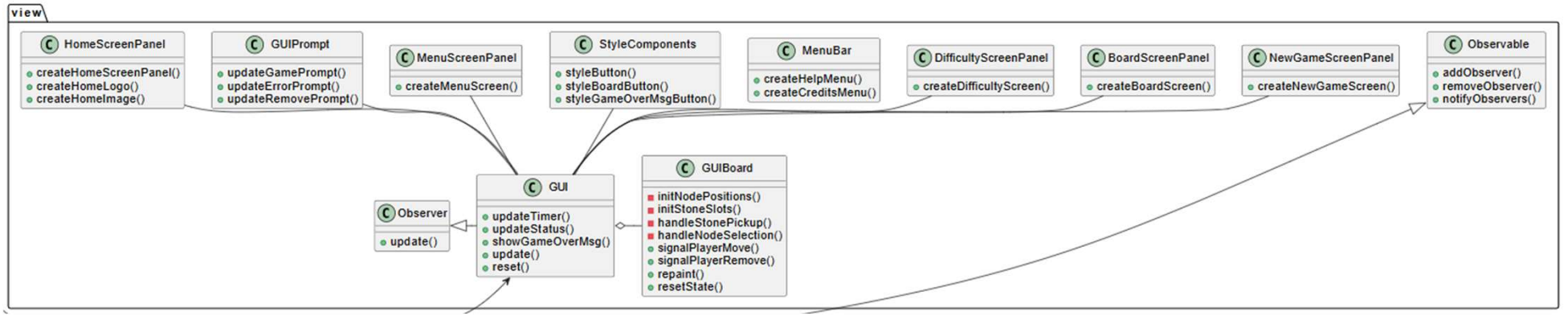
BoardLogic

Singletons

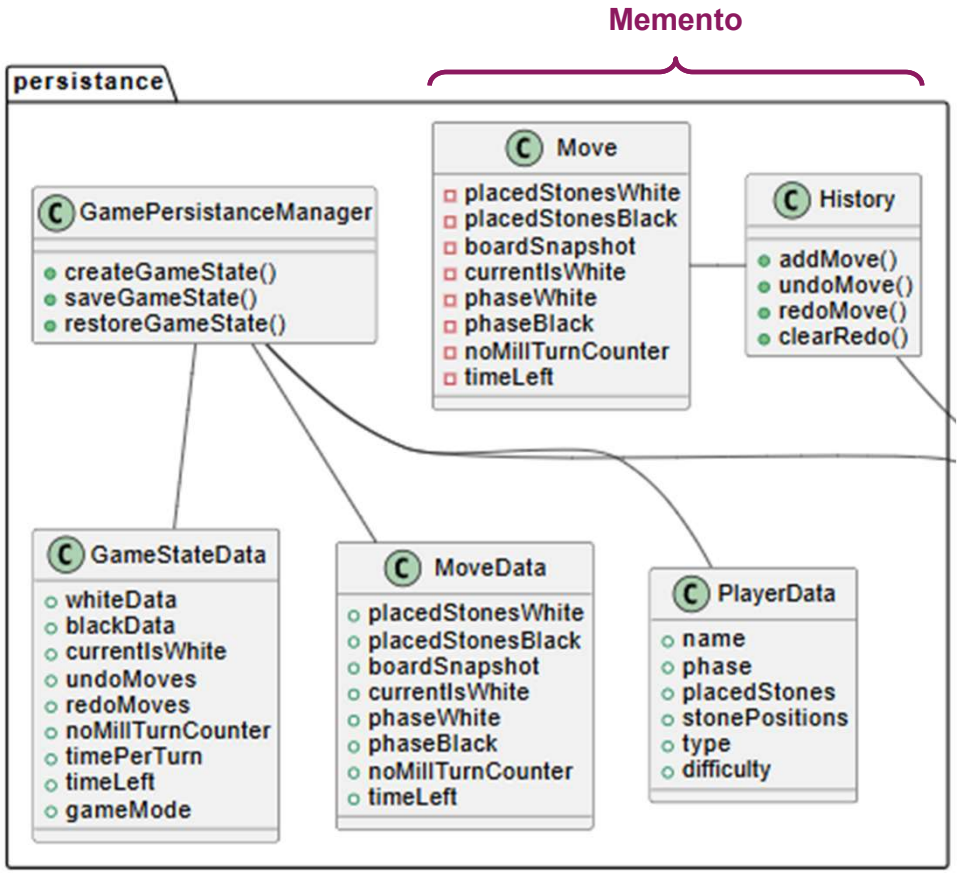


Architektur

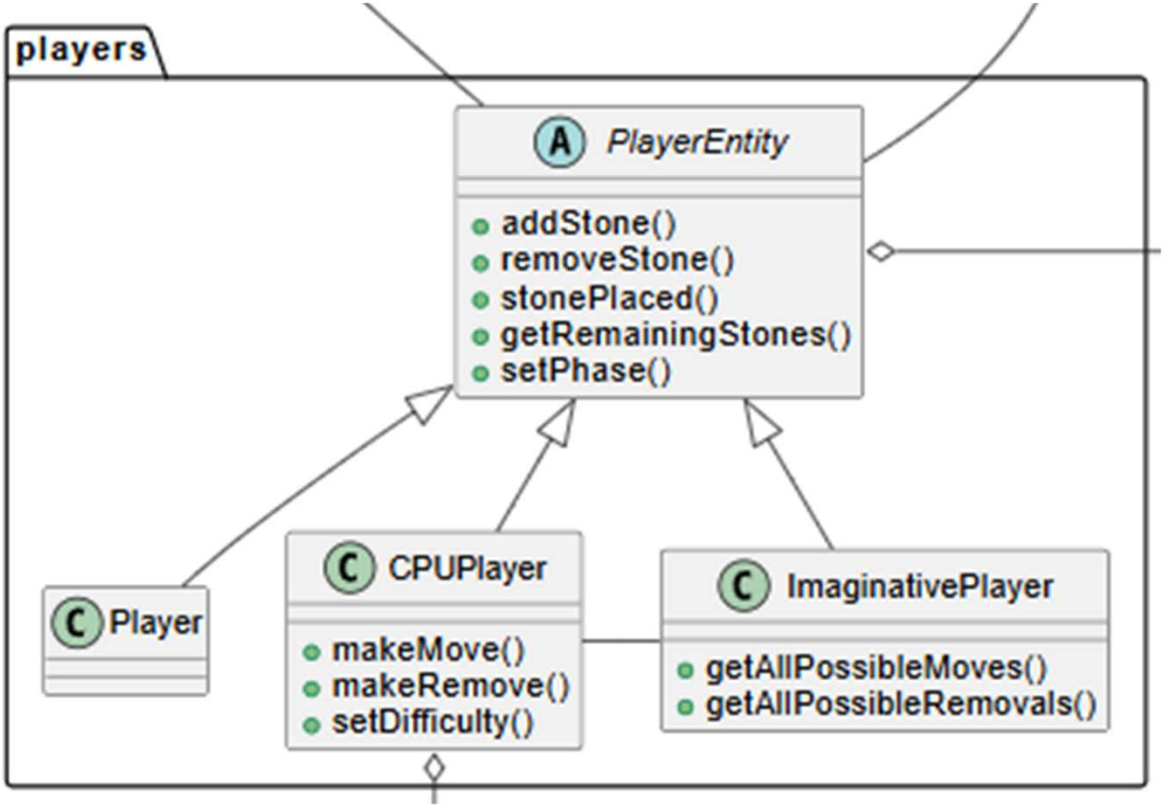
View



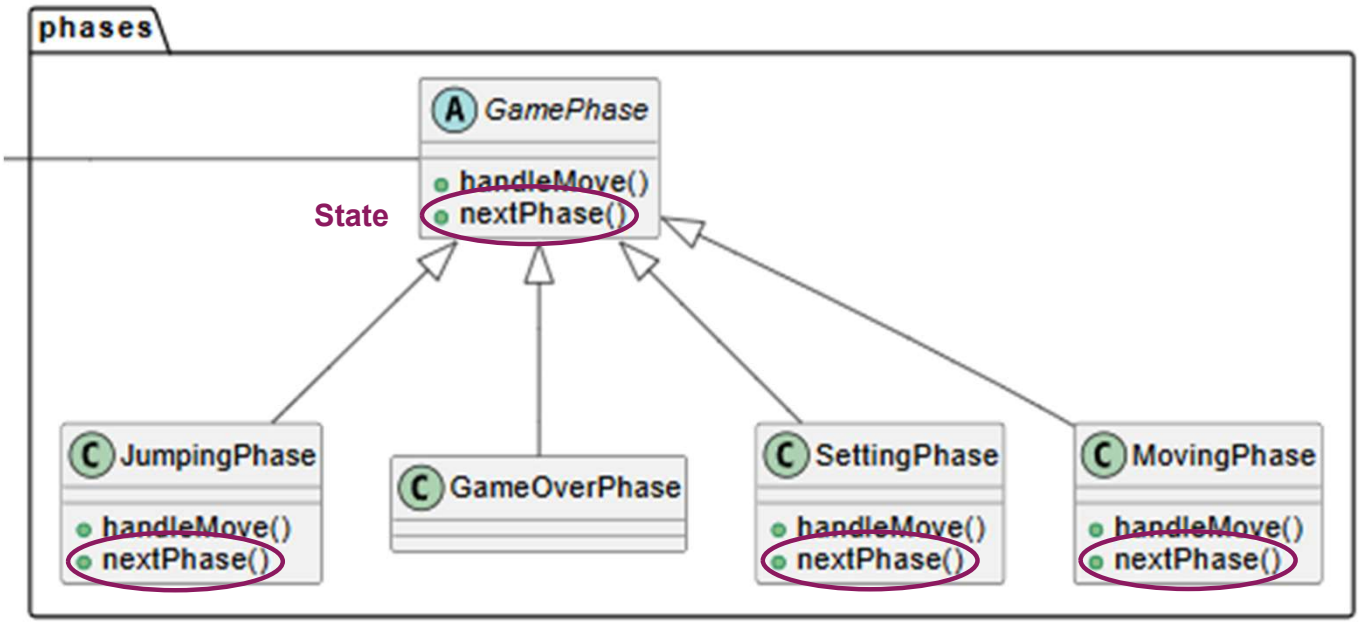
Persistance



Players

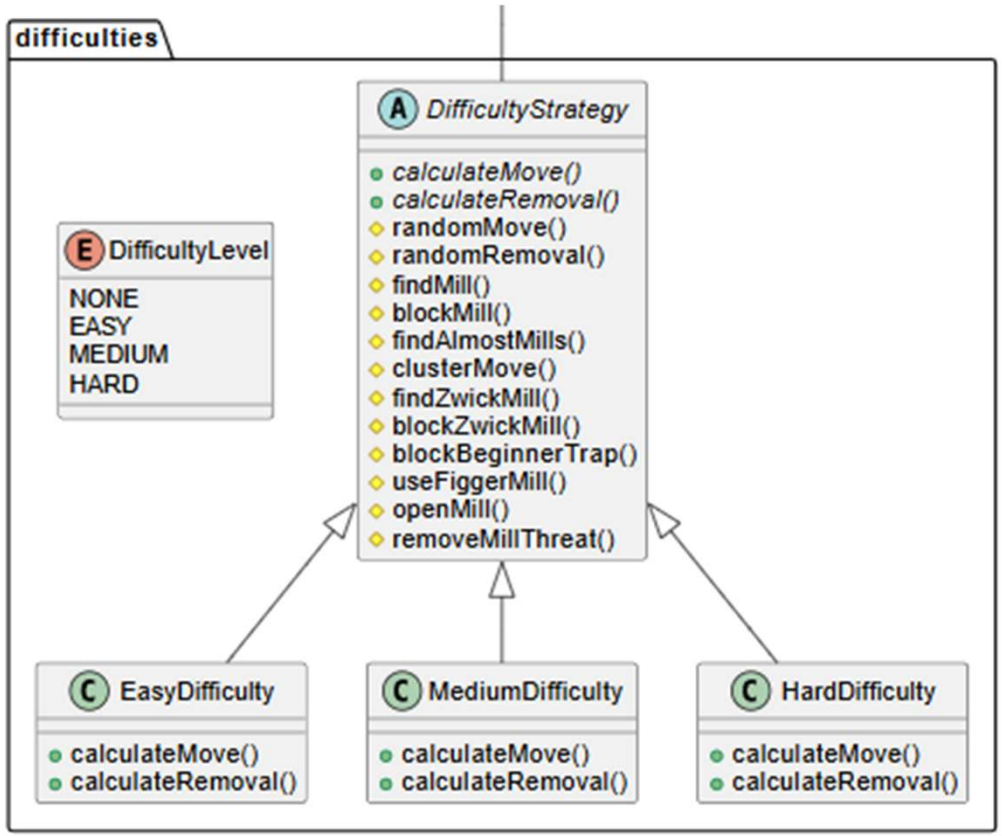

players
PlayerEntity
addStone()
removeStone()
stonePlaced()
getRemainingStones()
setPhase()
Player
CPUPlayer
makeMove()
makeRemove()
setDifficulty()
ImaginativePlayer
getAllPossibleMoves()
getAllPossibleRemovals()

Phases




```
public void nextPhase() {
    if (playerEntity.allStonesPlaced()) {
        playerEntity.setPhase(new MovingPhase(playerEntity));
```

Difficulties





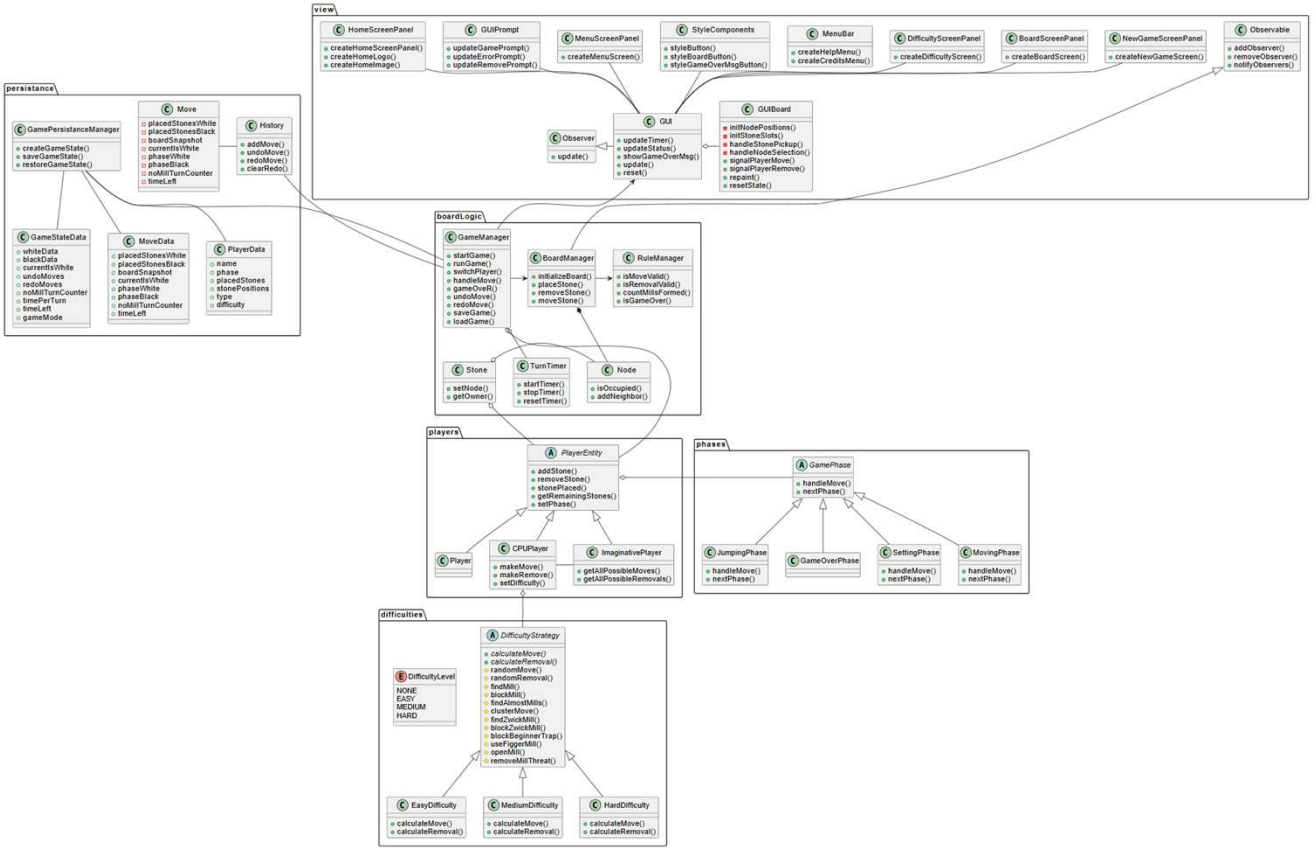
OST
Ostschweizer
Fachhochschule

KI-Gegner

WO WISSEN WIRKT.

KI-Gegner

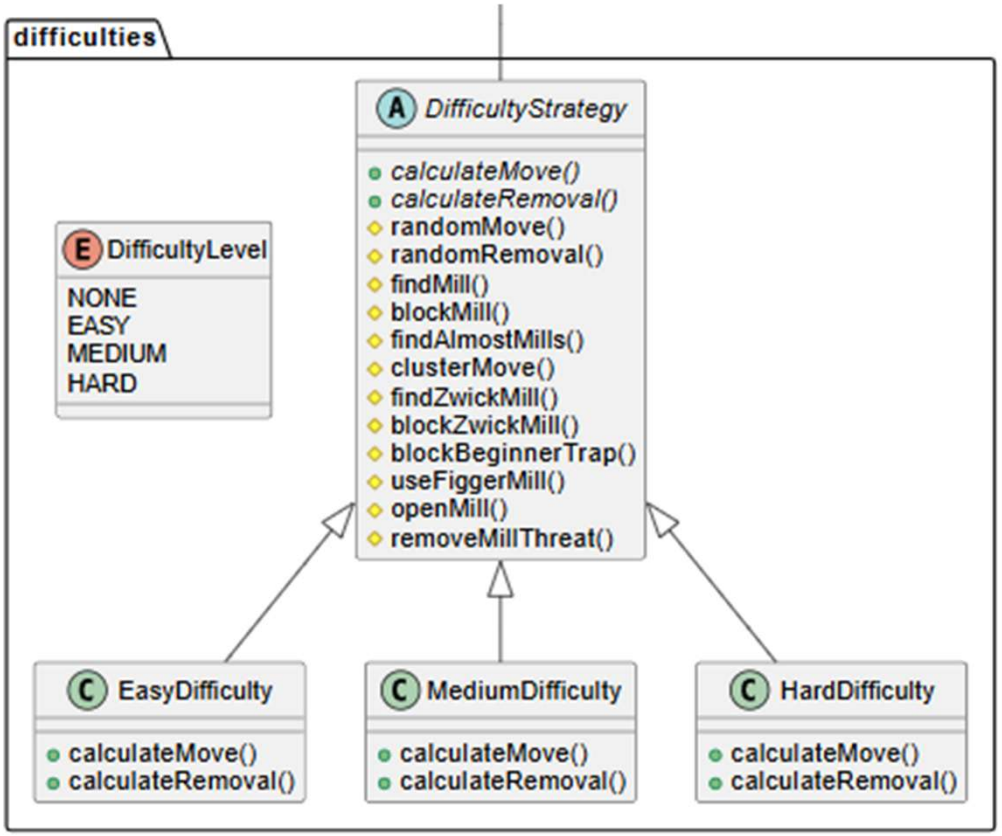
Übersicht (Vereinfacht)



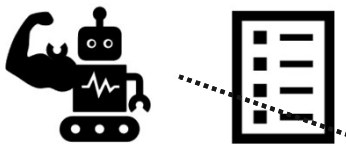
Difficulties

Aufgaben KI-Gegner:

- Eigenen Stein positionieren
- Gegnerischen Stein entfernen



Konzept





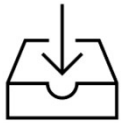
Pendelmühle



Mühle



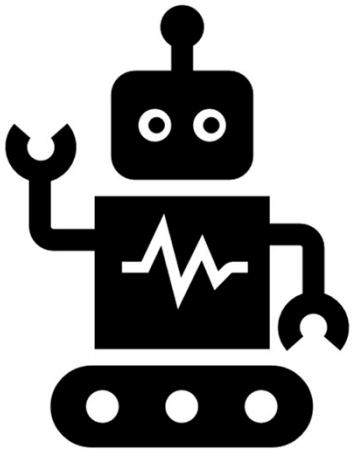
Mühle blockieren



Künftige Zwickmühle

Konzept

- Sequenzielles Suchen in Taktiken
- Reihenfolge definiert Priorität
- Unkonventionelle, entkoppelte «Chain of Responsibility»
- Schnelle Berechnung
- Nachvollziehbar
- 9 Taktiken für Spielzüge



KI-Gegner

Schwierigkeitslevel

Taktik	Verwendet in Phase	EasyDifficulty	MediumDifficulty	HardDifficulty
Pendelmühle verwenden	Ziehphase	-	-	1
Mühle machen	Alle Phasen	-	1	2
Mühle blockieren	Alle Phasen	-	2	3
Zwickmühle finden	Setzphase	-	-	4
Zwickmühle blockieren	Setzphase	-	-	5
Eckensymmetrie	Ziehphase	-	-	6
Stein in der Nähe setzen	Alle Phasen	-	3	7
Mühle öffnen	Zieh- & Sprungphase	-	-	8
Zufälliger Spielzug	Alle Phasen	1	4	9

Code: EasyDifficulty

```java
public class EasyDifficulty extends DifficultyStrategy {

    @Override
    public Node[] calculateMove(ImaginativePlayer player, ImaginativePlayer opponent) {
        List<Node[]> moves = player.getAllPossibleMoves();
        return randomMove(moves);
    }

    @Override
    public Node calculateRemoval(ImaginativePlayer player, ImaginativePlayer opponent) {
        List<Node> removals = player.getAllPossibleRemovals(opponent);
        return randomRemoval(removals);
    }
}
```

Code: MediumDifficulty

```java
public class MediumDifficulty extends DifficultyStrategy {

    @Override
    public Node[] calculateMove(ImaginativePlayer player, ImaginativePlayer opponent) {
        List<Node[]> playerMoves = player.getAllPossibleMoves();
        List<Node[]> opponentMoves = opponent.getAllPossibleMoves();

        return tryMoves(
                entry(name:"findMill", () -> findMill(playerMoves, player)),
                entry(name:"blockMill", () -> blockMill(playerMoves, opponentMoves, opponent)),
                entry(name:"clusterMove", () -> clusterMove(playerMoves, player)),
                entry(name:"randomMove", () -> randomMove(playerMoves)));
    }

    @Override
    public Node calculateRemoval(ImaginativePlayer player, ImaginativePlayer opponent) {
        List<Node> removals = player.getAllPossibleRemovals(opponent);
        List<Node[]> opponentMoves = opponent.getAllPossibleMoves();

        Node removalNode = removeMillThreat(removals, opponentMoves, opponent);
        if (removalNode != null) {
            return removalNode;
        }

        return randomRemoval(removals);
    }
}
```

KI-Gegner

Code: HardDifficulty

```java
public class HardDifficulty extends DifficultyStrategy {

    @Override
    public Node[] calculateMove(ImaginativePlayer player, ImaginativePlayer opponent) {
        List<Node[]> playerMoves = player.getAllPossibleMoves();
        List<Node[]> opponentMoves = opponent.getAllPossibleMoves();

        return tryMoves(
                entry(name:"useFiggerMill", () -> useFiggerMill(playerMoves, player)),
                entry(name:"findMill", () -> findMill(playerMoves, player)),
                entry(name:"blockMill", () -> blockMill(playerMoves, opponentMoves, opponent)),
                entry(name:"findZwickMill", () -> findZwickMill(playerMoves, player)),
                entry(name:"blockZwickmill", () -> blockZwickmill(playerMoves, opponentMoves, opponent)),
                entry(name:"blockBeginnerTrap", () -> blockBeginnerTrap(playerMoves, player)),
                entry(name:"clusterMove", () -> clusterMove(playerMoves, player)),
                entry(name:"openMill", () -> openMill(playerMoves, player, opponent)),
                entry(name:"randomMove", () -> randomMove(playerMoves)));
    }

    @Override
    public Node calculateRemoval(ImaginativePlayer player, ImaginativePlayer opponent) {
        List<Node> removals = player.getAllPossibleRemovals(opponent);
        List<Node[]> opponentMoves = opponent.getAllPossibleMoves();

        Node removal = removeMillThreat(removals, opponentMoves, opponent);
        return (removal != null) ? removal : randomRemoval(removals);
    }
}
```



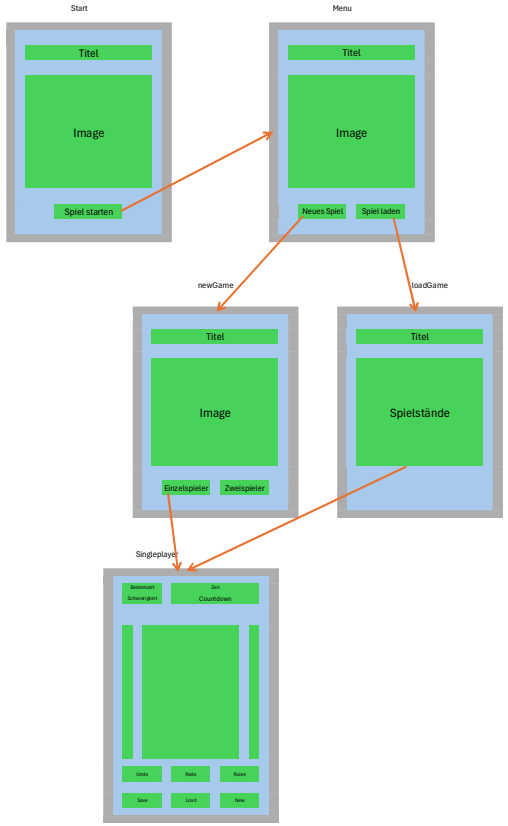
OST
Ostschweizer
Fachhochschule

Benutzeroberfläche

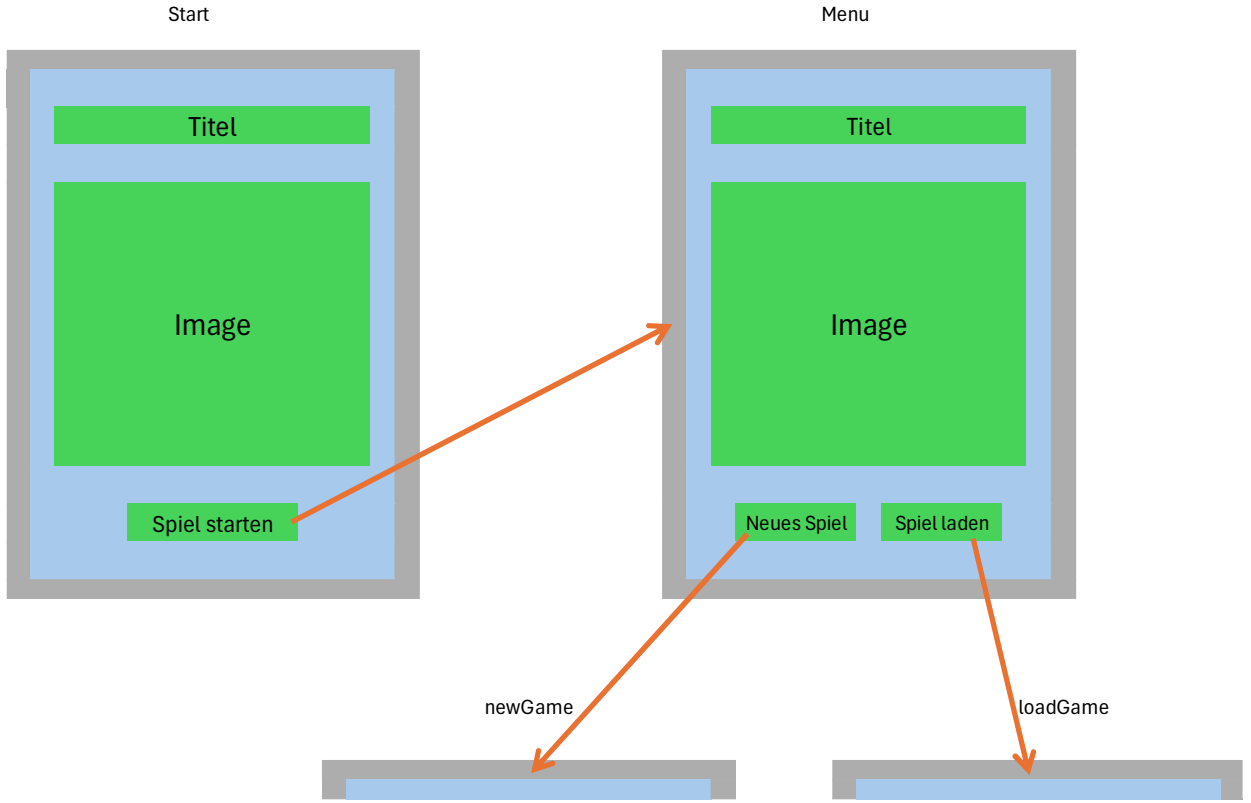
WO WISSEN WIRKT.

Benutzeroberfläche

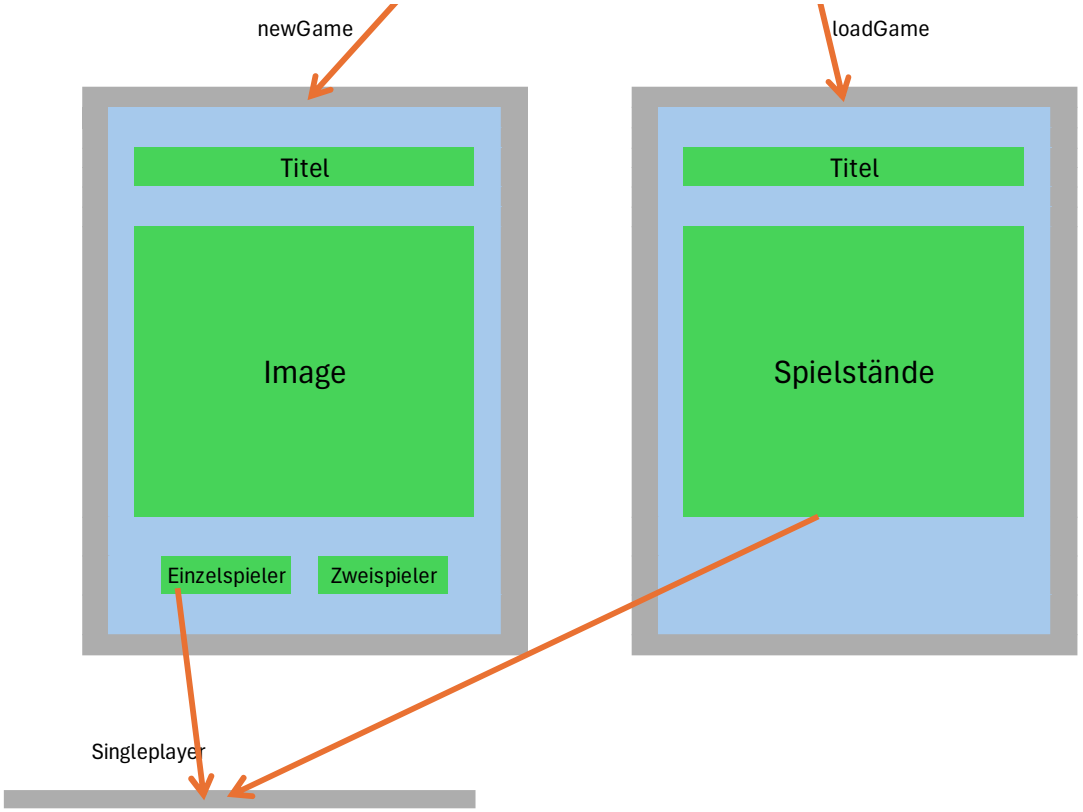
Konzept



Konzept

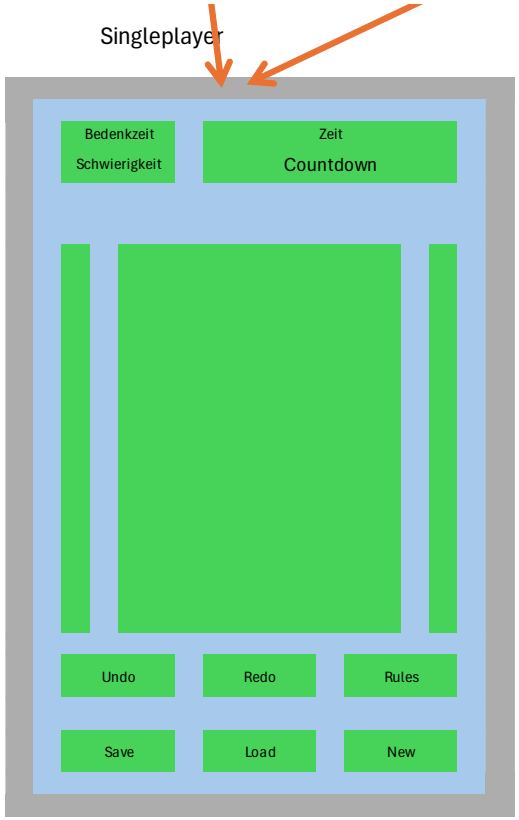

Start
Titel
Image
Spiel starten
Menu
Titel
Image
Neues Spiel
Spiel laden
newGame
loadGame

Konzept


newGame
loadGame
Titel
Image
Einzelspieler
Zweispieler
Titel
Spielstände
Singleplayer

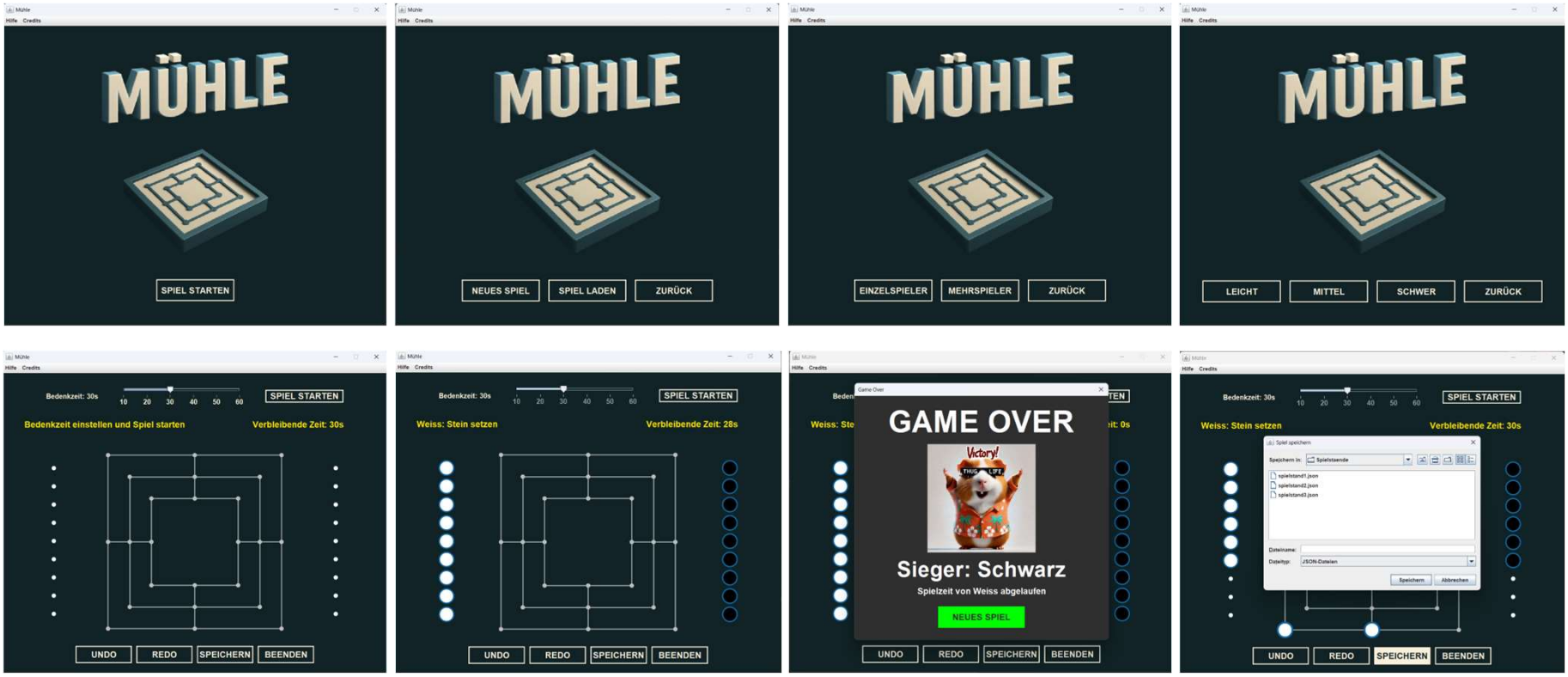
Benutzeroberfläche

Konzept

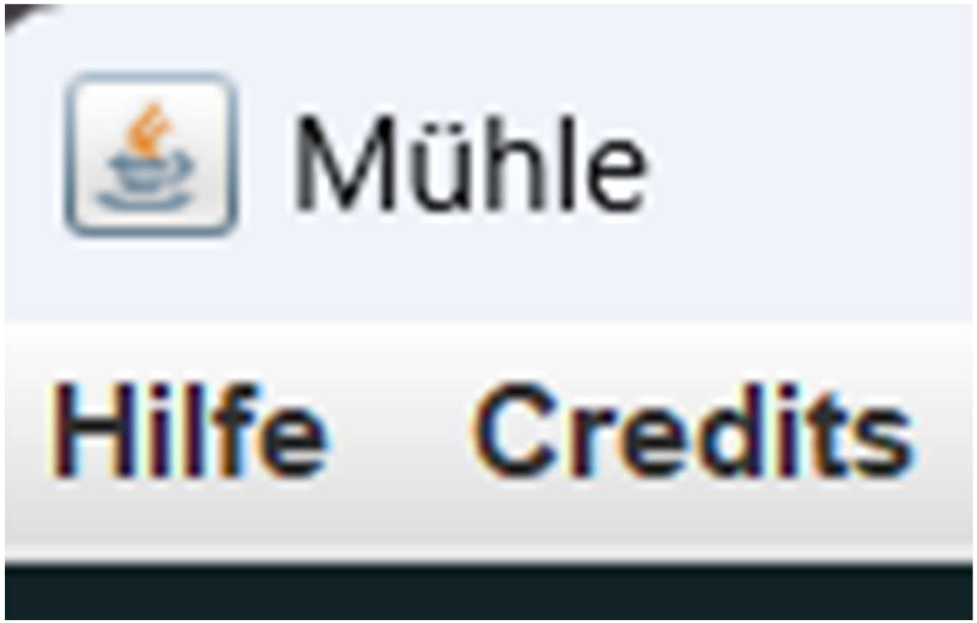


Benutzeroberfläche

Umsetzung



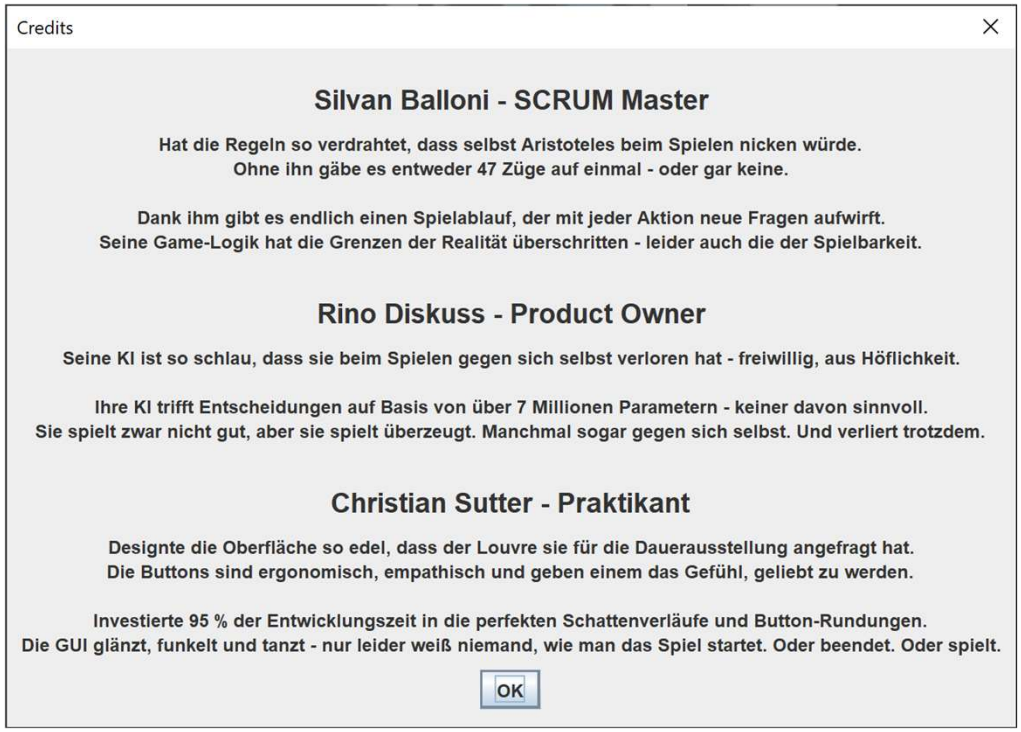
Umsetzung



Benutzeroberfläche

Umsetzung





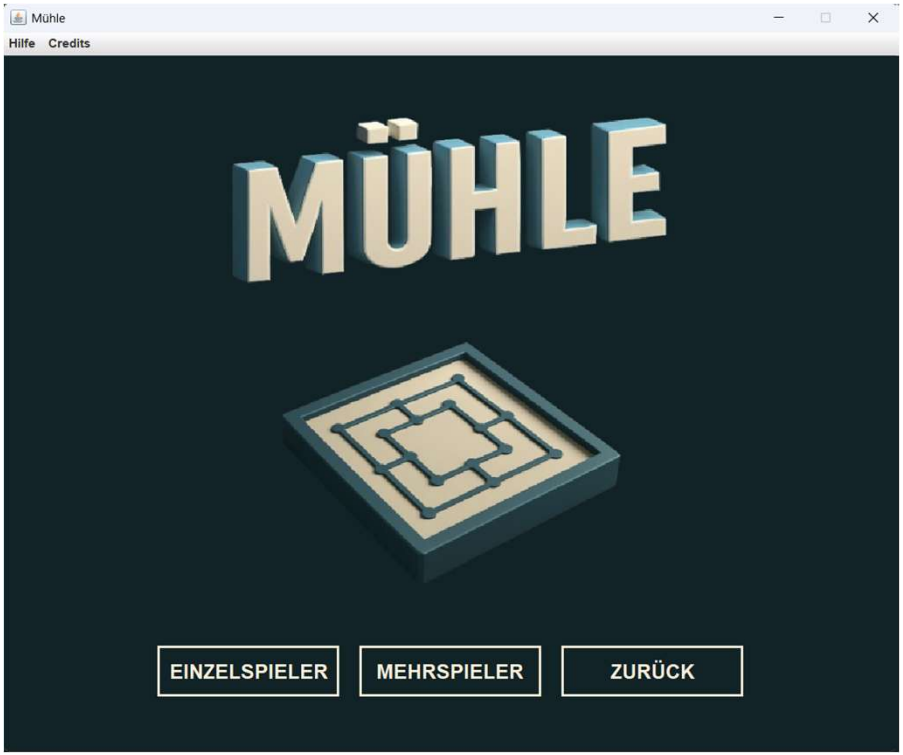
Benutzeroberfläche

Umsetzung



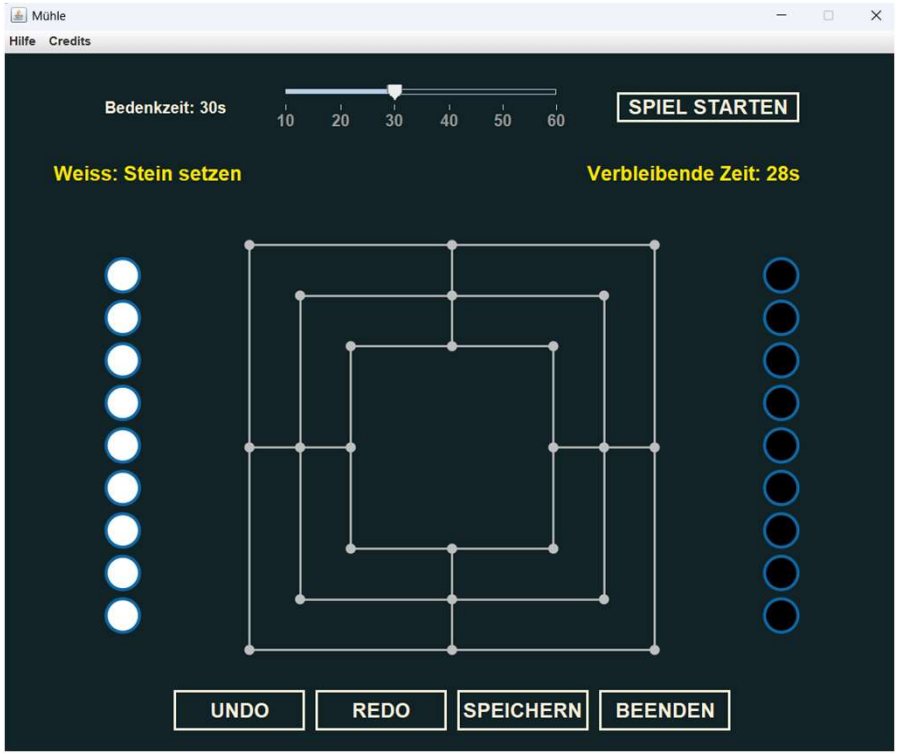
Benutzeroberfläche

Umsetzung



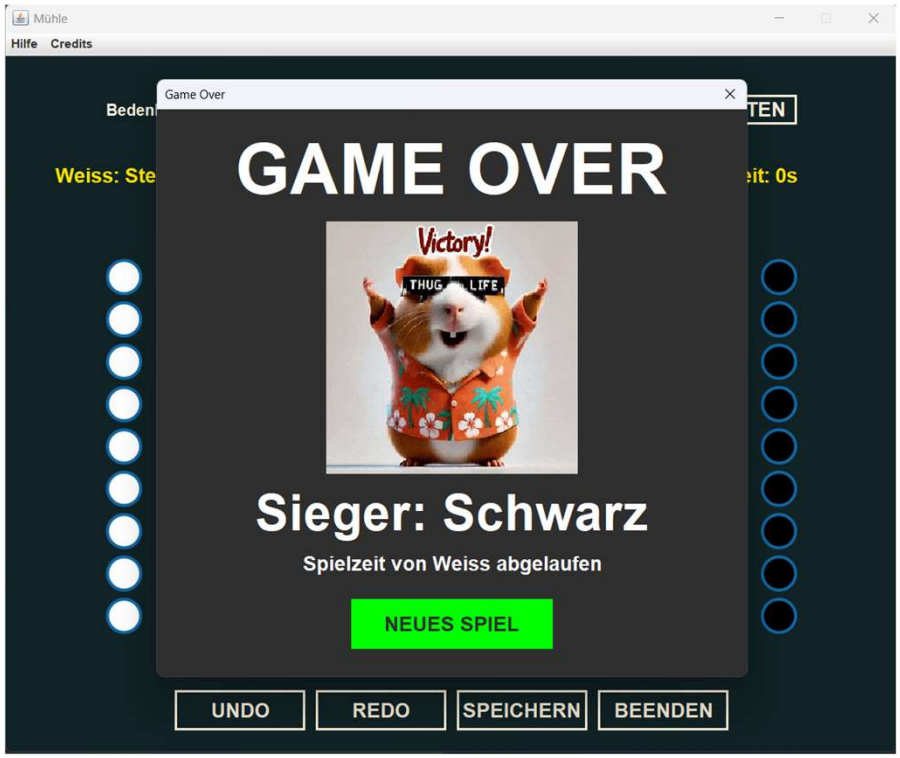
Benutzeroberfläche

Umsetzung



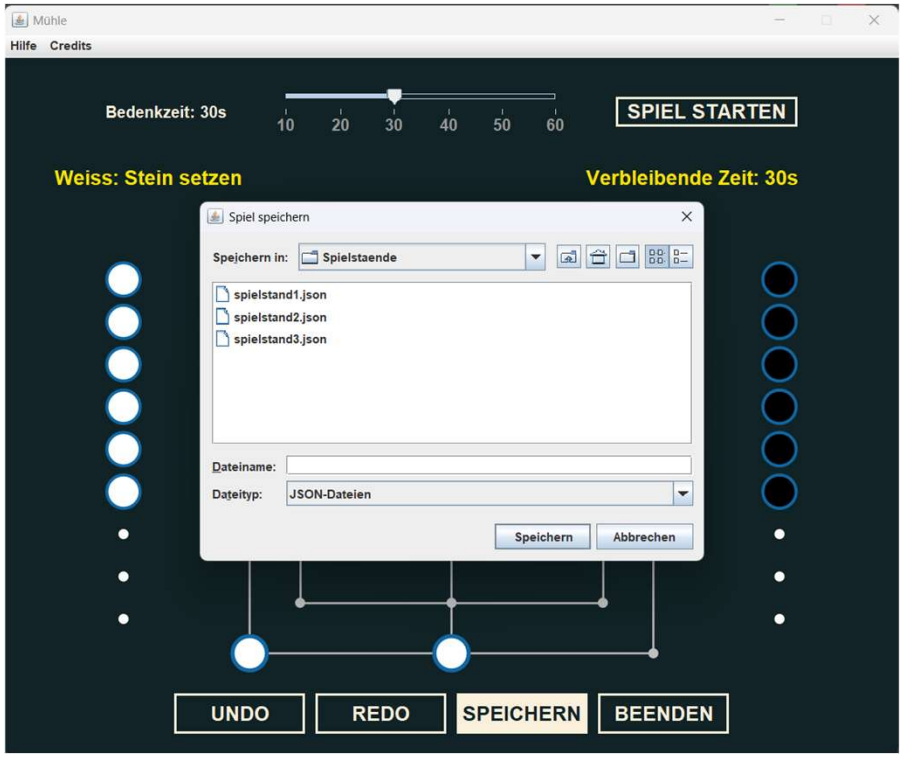
Benutzeroberfläche

Umsetzung



Benutzeroberfläche

Umsetzung





OST
Ostschweizer
Fachhochschule

Fazit

WO WISSEN WIRKT.

Fazit

Architektur

- MVC als Grundstruktur
- Zu Beginn gute Überlegungen und Struktur
- Mit der Zeit wurden Grenzen „verwässert“
- Metriken genzwertig

  → Refactoring notwendig
- Insgesamt übersichtlich und getrennte Zuständigkeiten

Fazit

KI-Gegner

Minimax-Algorithmus:

- war zu komplex → Funktionalität nicht gewährleistet
- Klassen: Flyweight, Adapter, GameTree, TreeNode

Sequenzielles Suchen in Taktiken:

- Einfach erweiterbar mit weiteren Taktiken oder Schwierigkeiten → variabel mit Zeitinvestition
- Saubere Schwierigkeitsklassen, komplexe Taktikenklasse
- Schwierigkeiten sind einfach anpassbar
- Schwierigkeiten sind beim Spielen bemerkbar unterschiedlich

Fazit

Benutzeroberfläche

Konzept:

- Sehr einfacher Aufbau durch Card Layout
- Navigation simpel über Button
- MVC – Pattern

Code:

- Design einfach anpassbar

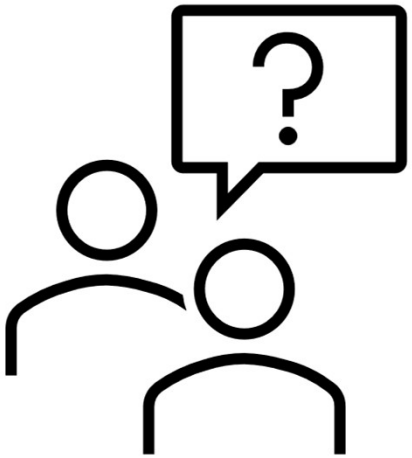
Bedienung:

- Drag & Drop nicht auf ersten Blick sichtbar
- Spiel starten nicht intuitiv
- Spielzug der KI nicht klar erkennbar

Organisation / Erfahrung

- Erstes SWE-Projekt
- SCRUM
- GIT-Probleme
- Bug mit der Memento-Implementierung
- Mehr Reviews wären nötig gewesen
- Zeitaufwand

Fragen





OST
Ostschweizer
Fachhochschule

Demonstration

WO WISSEN WIRKT.